



MIGRATION ROS 2 & ÉVOLUTION DU TER ATELIER FLEXIBLE

Vérification de la migration logicielle et implémentation de l'évacuation multi-poste

Samy Janati • Stage 4A SRI — UPSSITECH • LAAS-CNRS, équipe ROC

Encadré par Sandra Ulrich Ngueveu (LAAS-CNRS / ENSEEIHT) et Diane Bury (MFJA)

08/06/2026 — 31/08/2026

PLAN

- 1 **Contexte : Lieu d'accueil et support du stage**
- 2 **Objectif 1 : Valider la migration ROS1 vers ROS2**
- 3 **Objectif 2 : Implémenter l'évacuation par n'importe quel poste de travail**
- 4 **Démonstration**
- 5 **Bilan**

CONTEXTE : Lieux d'accueil

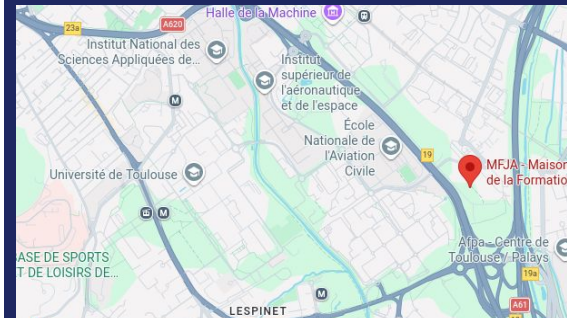
Le LAAS-CNRS

- 12 semaines
- Encadré par Sandra Ulrich Ngueveu
- Equipe ROC (Recherche Opérationnelle, Optimisation Combinatoire)



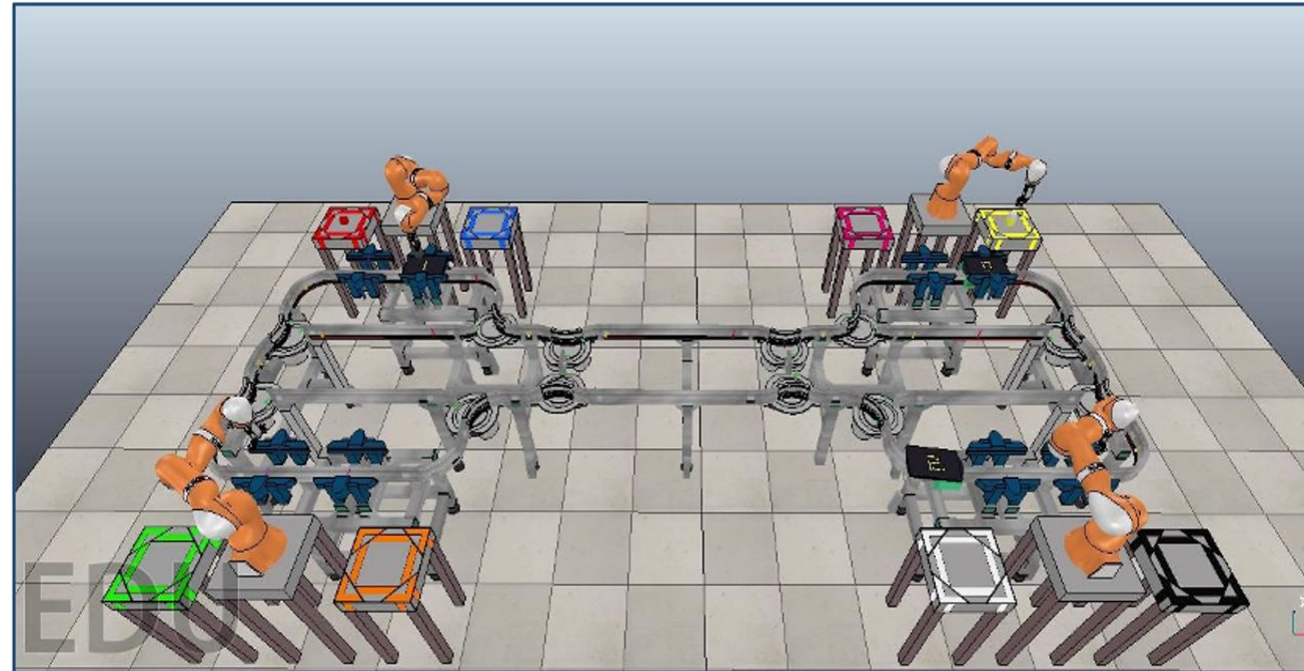
La MFJA (Maison de la Formation Jacqueline Auriol)

- Plateforme d'enseignement mutualisée Toulousaine
- Equipements industriels reels

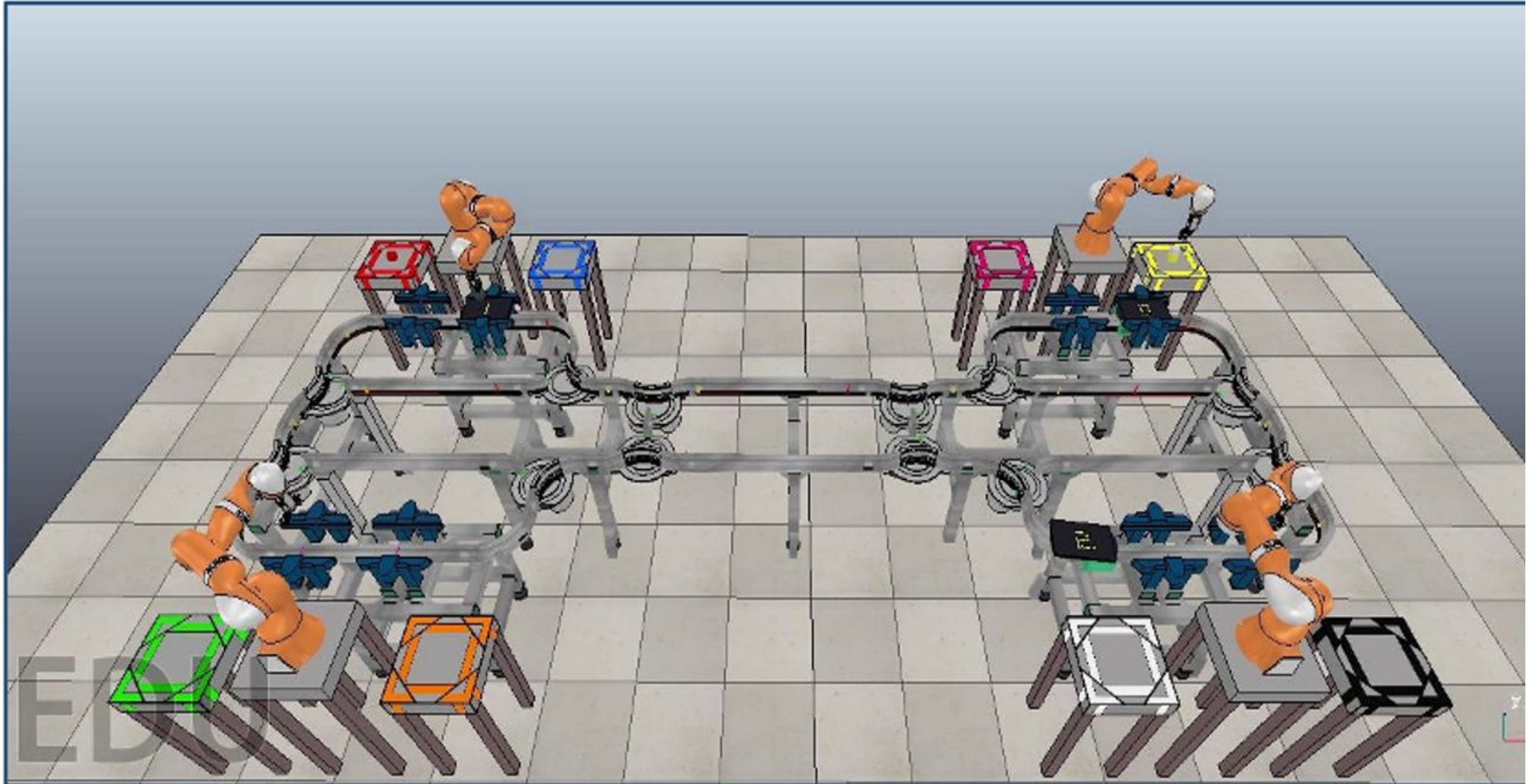


CONTEXTE : Le support du stage

- Cours de S. Ngueveu à l'ENSEEIH
 - Ordonnancement ; Planification ; SED
 - TER atelier flexible : commander et optimiser une cellule de production automatisée via un **réseau de Petri**.
- Version Physique à la MFJA
- Version simulée : CoppeliaSim + ROS



CONTEXTE : Le support du stage



Contexte

- ROS 1 obsolète depuis mai 2025, incompatible avec Ubuntu 24.04
- Un binôme précédent a migré le projet vers ROS 2 (nouveau standard)

OBJECTIF 1

Tester, finaliser et valider la migration

S'assurer que la simulation ROS 2 se comporte comme la version ROS 1, via des scénarios de production complets.

ROS
ROS2

OBJECTIF 2

Rendre l'évacuation possible par n'importe quel poste de travail

Jusqu'ici seul le poste 3 permettait d'évacuer un produit. Objectif : le rendre possible depuis les 8 postes.

OBJECTIF 1 : Valider la migration de ROS1 vers ROS2

Le simple démarrage de la simulation ne teste pas toutes les fonctionnalités , il fallait des scénarios de production complets (aiguillages, navettes, robots, évacuation).

1

VM Ubuntu 20

ROS 1 en parallèle
pour comparer
chaque
comportement

2

Test du code de tutoriel de lancement

Cas le plus classique

3

Test de code étudiant robuste + code personnel

Tester toutes les
fonctions haut niveau

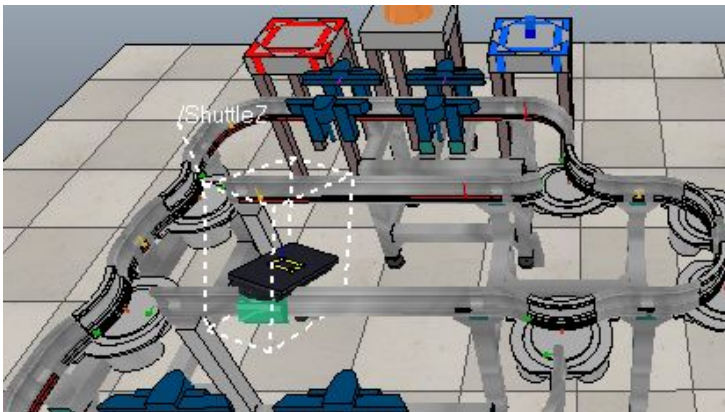
OBJECTIF 1 : Valider la migration de ROS1 vers ROS2

Une navette qui déraile... pourquoi ?

Problème : sous ROS 2, un aiguillage tourne exactement quand la navette passe dessus → chute. Le même code fonctionne parfaitement sous ROS 1.

Cause réelle

ROS 2 sous Ubuntu 24 à des temps d'exécutions plus rapides : la navette repart avant que l'aiguillage ait fini de se positionner.



La correction

13 s → **3 s**

Durée de tâche ajustée dans le fichier de configuration → comportement à nouveau conforme.

OBJECTIF 1 : Résultats des tests de migration

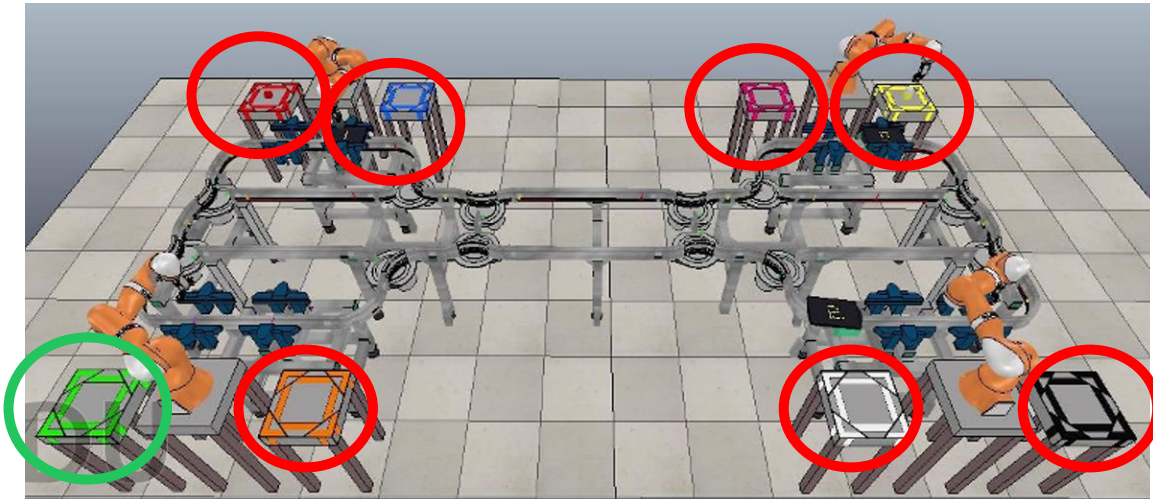
Code testé	Résultat initial	Après ajustement
Tutoriel de Lancement	Échec (collision)	Succès
Code etudiant robuste	Succès	—
Code personnel et test du reste des fonctions haut niveau	Succès	—

=> La migration est validée pour les codes les plus robustes

OBJECTIF 2 : Rendre l'évacuation possible par n'importe quel poste de travail

Demande d'évacuation initiale : `robot.Evacuer()`

→ évacue toujours le poste 3, quel que soit le contexte.



Seul le poste 3 (en vert) permettait l'évacuation

Objectif

`robot.Evacuer(numero_poste)`

Le poste devient un paramètre choisi par l'étudiant

OBJECTIF 2 : Rendre l'évacuation possible par n'importe quel poste de travail

Comprendre la chaîne avant de la modifier



Ce que j'ai découvert

Le message d'évacuation était vide et le code contenait un test figé avec la position toujours fixée au poste 3.

OBJECTIF 2 : Rendre l'évacuation possible par n'importe quel poste de travail

Choix d'implémentation

1

Diffusion à tous

Le message est envoyé aux 4 robots, chacun filtre selon son numéro ce qui est cohérent avec le reste de l'architecture du projet.

2

Message dédié (nouveau type)

Un type de message , contenant un seul entier, modulaire pour ajouter d'autres champs plus tard.

3

Rétro-compatibilité

Evacuer(int numero_poste = 3) : les anciens codes continuent de fonctionner sans aucune modification.

OBJECTIF 2 : Rendre l'évacuation possible par n'importe quel poste de travail

Enregistrer, puis vérifier

Le log

Rôle : constater les faits, résultat de la production de l'étudiant

Le numéro de poste est ajouté dans le log



Le checker

Rôle : juger la conformité.

Programme Python indépendant, lancé après la simulation : compare le log au fichier de configuration.



La gamme de fabrication

Rôle : cahier des charges, la production voulue

Ajout : vérifie que le poste évacué appartient bien aux postes autorisés erreur bloquante sinon.

Validation à trois niveaux

Compatibilité

Evacuer() et Evacuer(3)

Toujours poste 3 ✓

Multi-poste

8 postes testés

8 / 8 conformes ✓

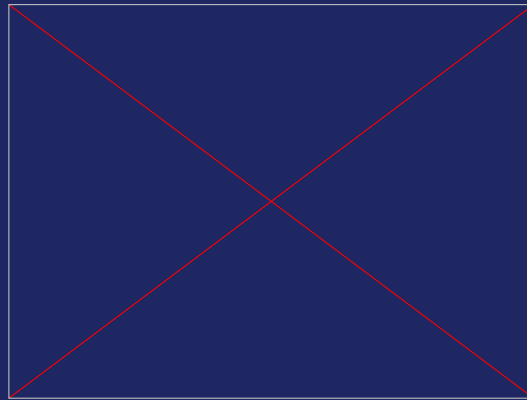
Checker

8 postes × 2 configs

16 / 16 conformes ✓

```
1
Pause pour synchronisation RDS 2...
Marquage : M[0]=1,
Evacuation du poste 8 !
Marquage : M[1000]=1,

—[TEST DE VALIDATION TERMINE AVEC SUCCES]—
```



DÉMONSTRATION

Évacuation d'un produit depuis un poste autre que le poste 3, avec vérification par le checker

PERSPECTIVES

Et après ?

Pont avec la maquette réelle

Lier les commandes ROS 2
développées sous CoppeliaSim aux
automates réels de la MFJA.

Choix libre de l'emplacement de départ des navettes

Piste explorée du binôme précédent : laisser
l'utilisateur choisir ses navettes au
démarrage. Fonction clé déjà localisée, reste
l'interaction utilisateur.

BILAN

Compétences et retour d'expérience

- 1 **Développement** C++ sur base réelle, programmation orientée objet
- 2 **Robotique** ROS 2, robotique industrielle
- 3 **Méthodologie** Tests isolés, Git, décomposition en étapes

Ce stage a confirmé mon envie de m'orienter vers la robotique industrielle et les bras manipulateurs pour la suite de mon parcours.

CONTRIBUTION

1. Création du message (`robots/msg/EvacuerMsg.msg`)

Un nouveau type de message ROS, contenant juste un entier (`numero_poste`), déclaré dans le package `robots`.

2. Modification de `RobotsInterface` (package `commande`)

La fonction `Evacuer()` devient `Evacuer(int numero_poste = 3)` : elle construit le nouveau message avec le poste demandé et le publie, au lieu d'un message vide.

3. Modification de `Robot` (package `robots`)

Le callback qui écoute le topic est adapté au nouveau type de message. La fonction `Evacuer()` est réécrite avec les 8 `if` (poste → robot + position), et ne déclenche l'évacuation que si le robot courant est concerné.

4. Enrichissement du log (`Robot.cpp` + `LogManager.cpp`)

Le numéro de poste est ajouté en tête du tableau envoyé au log, et écrit dans `log.txt`

5. Mise à jour du checker (`checker.py`) + config (`dataprodconfig`)

Ajout de la variable `Postes_evacuation` dans la config, et modifications du checker pour lire le poste dans le log et vérifier qu'il est autorisé.

Fonctions disponibles pour les étudiants

Aiguillages

- `aiguillage.Gauche(num) / aiguillage.Droite(num)`

Navettes

- `cmd.Stop_PS(num) / cmd.Ouvrir_PS(num)`
- `capteur.get_PS(num), get_CP(num), get_CPI(num), get_DG(num), get_DD(num)`

Robots

- `robot.DeplacerPiece(robot, posA, posB) / robot.FinDeplacerPiece(robot)`
- `robot.FaireTache(poste, duree) / robot.TacheFinie(poste)`
- `robot.AjouterProduit(poste, type)`
- `robot.Evacuer()` → **`robot.Evacuer(numero_poste)`**